
Why Don't Software Developers Use Static Analysis Tools to Find Bugs?

ESE
Sep. 4th, 2019
João Paulo

Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and
Robert Bowdidge - ICSE 2013

Introduction

Motivation

- Different ways to ensure software quality
 - code reviews
 - rigorous testing
- Software defects can cost companies significant amounts of money
- Static analysis tools
 - assess source code
 - **without running it**
 - help ensure **higher quality** software
 - through the **development process**

Motivation

- Static analysis tools can run
 - integrated with developer's environment
 - from the command line
 - as a separate tool
- Static analysis tools can provide
 - defects list
 - defects severity
 - warnings about defects
 - suggested solutions (*quick fixes*)
- Saving **time** and **cost**

Static analysis tools drawbacks

- dealing with their interface became a **burden**
- finding the warnings is **not easy**
- hard time to **interpret** the feedback
- **confusion**

Static analysis tools previous studies

- There have been studies about static analysis tools that
 - assess their **correctness**
 - analyses their **functionalities**, and
 - how developers **use** them
- None concerns on what tools **do** or **can do** for a developer
 - or are based on the developers' **perspective**

Current study

Focused on tools used to **find bugs**

Aims to **understand**

- **Why** developers are not using static analysis tools
- **How** current tools can be improved

Is **based on**

- developers' **perceptions** on using such tools, and
 - what may **cause** that perceptions
-

—

"Perception plays an important role when considering human and computer interactions"

Related work

3 works reported

Ayewah and Pugh

- tools should help engineers find bugs as **early** as possible

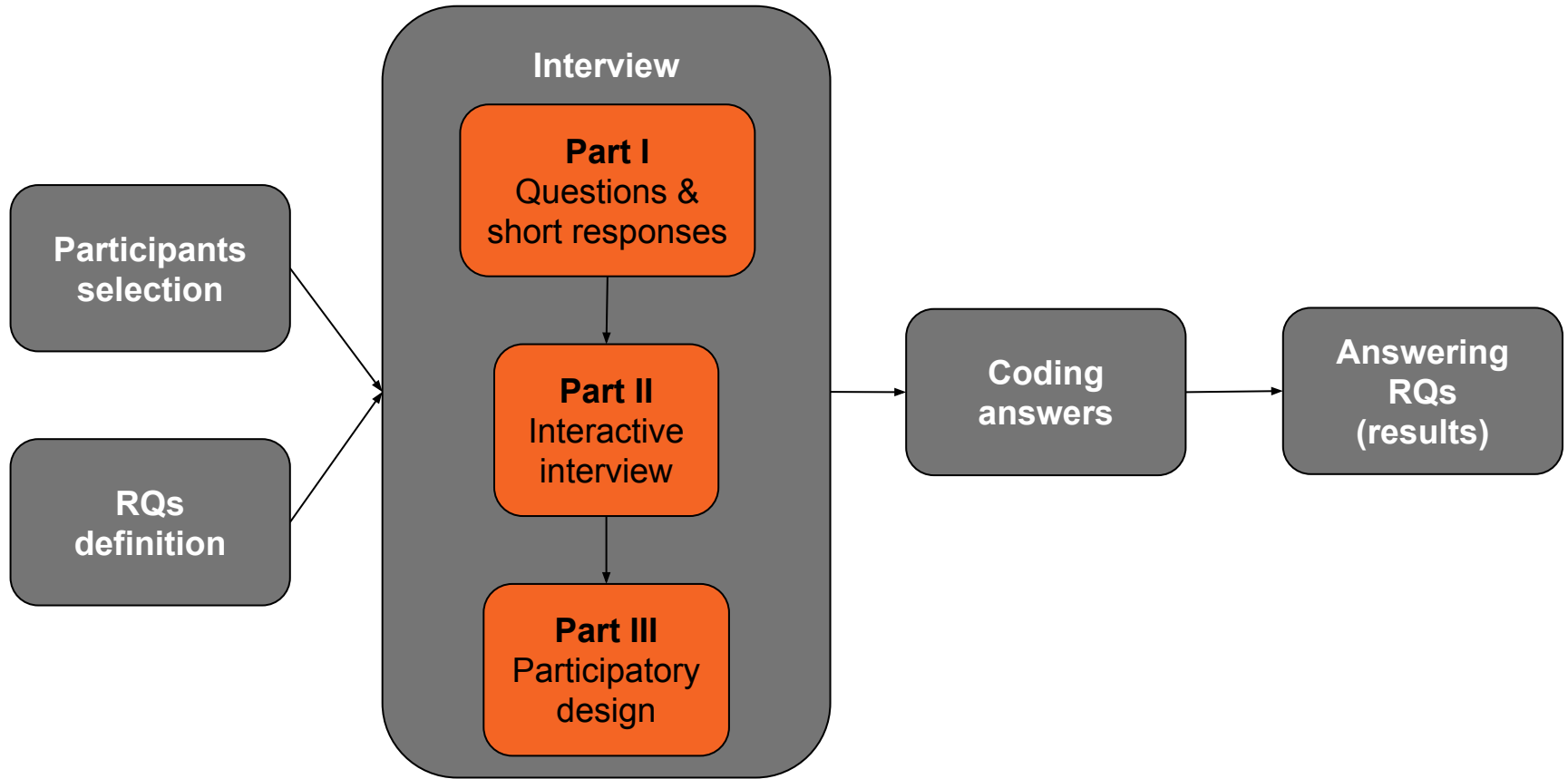
Khoo et al.

- *Path projection*: UI that allow navigation through message errors

Heckman and Williams

- FAULTBENCH: benchmark to compare static analysis tools

Methodology



Participants selection

20 developers that **currently uses** or **have used** static analysis tools

- 16 **professional developers** at a large company
- 4 **graduate** students at NCSU with previous experience

Recruited by an electronic recruitment flyer

- first sent to authors' contacts in industry
- that then sent to developers in their company

The participants

Participant	Open-source Tools	Closed-source Tools	Local
Abby	FindBugs	IntelliJ	Yes
Adam	CheckStyle, FindBugs, PMD	IntelliJ	Yes
Andy	FindBugs, Lint	Jtest [20]	Yes
Chris	CheckStyle, FindBugs, Lint	Coverity	Yes
Cody	Dehydra	-	Yes
Frank	-	-	Yes
Gordon	Lint, CheckStyle, FindBugs	-	Yes
Jake	FindBugs, Lint	FlexLint, Klocwork Insight [21], Visual Studio [22]	Yes
James	Lint, CheckStyle, FindBugs	Visual Studio	Yes
Jason	Lint, FindBugs	-	Yes
John	CheckStyle, Copy/Paste Detector(CPD), FindBugs, Lint, PMD	CodePro [23]	Yes
Jordan	CheckStyle, FindBugs, PMD	Jtest	Yes
Josh	FindBugs, Lint	Coverity	No
Lee	CheckStyle, FindBugs, Lint	Visual Studio	Yes
Matt	Lint	FlexLint, PyCharm	Yes
Mike	cpplint, Lint	-	Yes
Phil	-	-	Yes
Ray	CheckStyle, FindBugs	-	Yes
Ryan	FindBugs, Lint	Coverity	Yes
Steve	CheckStyle, CPD, FindBugs, Lint	IntelliJ	Yes
Tony	CPD, FindBugs, Lint, Splint,cpplint, PMD, Checkstyle	Coverity	No

The participants

Participant	Open-source Tools	Closed-source Tools	Local
Abby	FindBugs	IntelliJ	Yes
Adam	CheckStyle, FindBugs, PMD	IntelliJ	Yes
Andy	FindBugs, Lint	Jtest [20]	Yes
Chris	CheckStyle, FindBugs, Lint	Coverity	Yes
Cody	Dehydra	-	Yes
Frank	-	-	Yes
Gordon	Lint, CheckStyle, FindBugs	-	Yes
Jake	FindBugs, Lint	FlexLint, Klocwork Insight [21], Visual Studio [22]	Yes
James	Lint, CheckStyle, FindBugs	Visual Studio	Yes
Jason	Lint, FindBugs	-	Yes
John	CheckStyle, Copy/Paste Detector(CPD), FindBugs, Lint, PMD	CodePro [23]	Yes
Jordan	CheckStyle, FindBugs, PMD	Jtest	Yes
Josh	FindBugs, Lint	Coverity	No
Lee	CheckStyle, FindBugs, Lint	Visual Studio	Yes
Matt	Lint	FlexLint, PyCharm	Yes
Mike	cpplint, Lint	-	Yes
Phil	-	-	Yes
Ray	CheckStyle, FindBugs	-	Yes
Ryan	FindBugs, Lint	Coverity	Yes
Steve	CheckStyle, CPD, FindBugs, Lint	IntelliJ	Yes
Tony	CPD, FindBugs, Lint, Splint,cpplint, PMD, Checkstyle	Coverity	No

Open-source

static analysis tools

1. FindBugs
2. CheckStyle
3. PMD
4. Lint
5. Dehydra
6. Copy/Paste Detector (CPD)
7. cpplint
8. Splint

Closed-source

static analysis tools

1. IntelliJ
2. Jtest
3. Coverity
4. FlexLint
5. Klocwork Insight
6. Visual Studio
7. CodePro
8. PyCharm

Research questions

RQ1: What reasons do developers have for using or not using static analysis tools to find bugs?

RQ2: How well do current static analysis tools fit into the workflows of developers?

RQ3: What improvements do developers want to see being made to static analysis tools?

Interviews - Part I

Questions and short responses

Aiming at answering RQ1

Questions about:

- **first** experience
- anything that **stood out** (as easy or difficult)
- ever used a tool in **team setting**
- **consciously avoided** a tool
- **critical** characteristics of a good tool



Interviews - Part II

Interactive interview

Aiming at answering RQ2

The authors **observed** developers actually using a static analysis tool.



- got more detailed information on **how** was the usage
- asked to developers to explain their steps **out loud** with questions like:
 - what are your **next move(s)**?
 - how do you **configure** settings from default?
 - does the tool **aids in assessing** what to do about a warning?
 - are **quick fixes** or code suggestions **helpful** (when available)?

Interviews - Part III

Participatory design

Aiming at answering RQ3

Blank sheet of paper

Instead of saying, the participants were asked to show:

- what they want a tool to look like
- how it should work

Curiosity:

- 6 drew something
- The remaining provided verbal descriptions



RQ3:
improvements

Coding

interview responses

Tool output

User input/customizability

Supporting teamwork

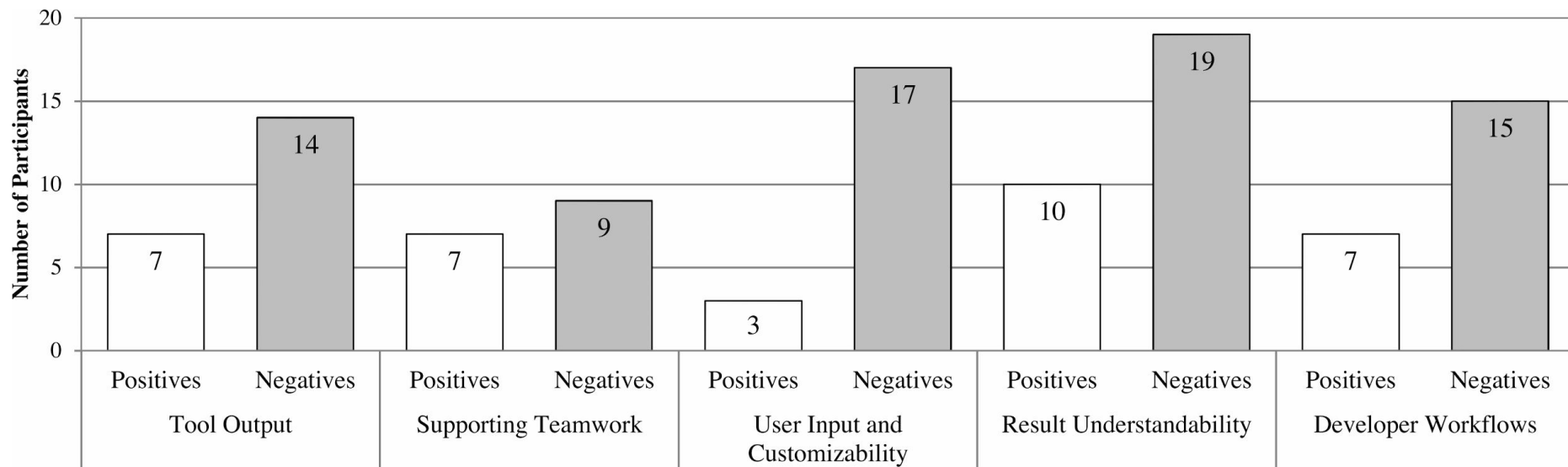
Result understandability

Workflows

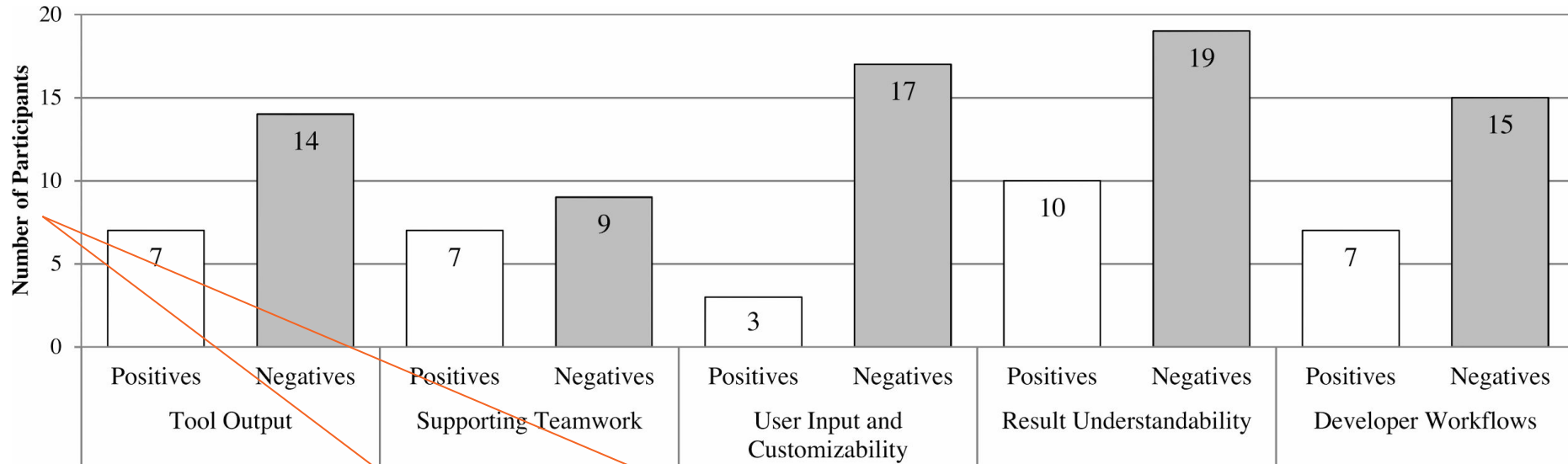
Tool design

Results

Good and bad about tools used



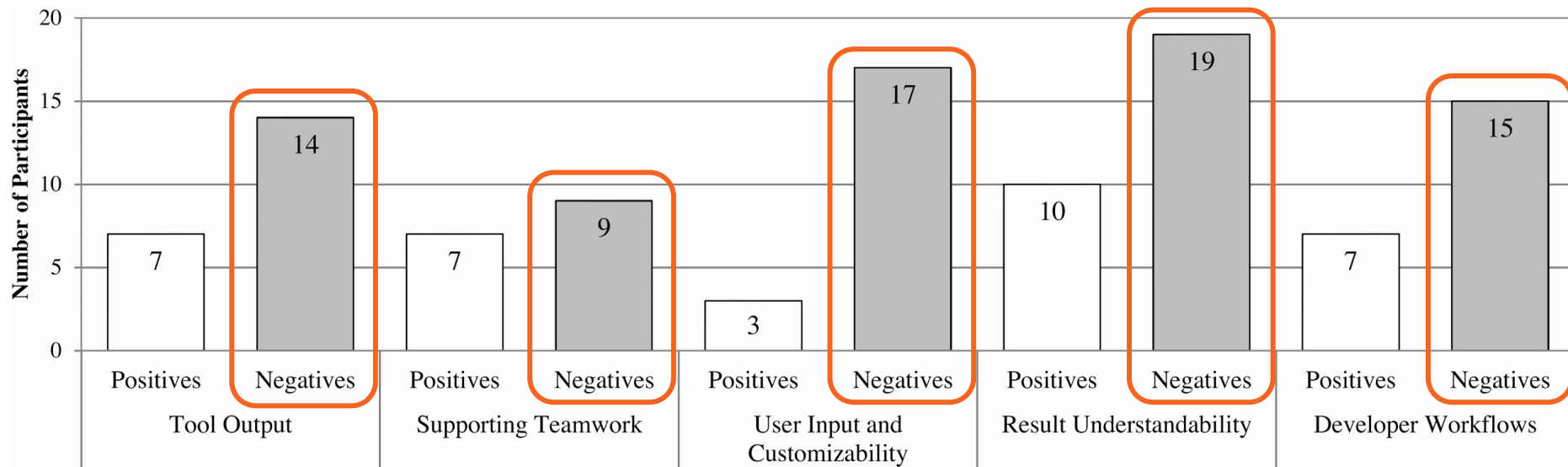
Good and bad about tools used



Each bar could totalize 20.

For each good or bad thing a participant said about any aspect of the tool s/he used, the authors computed 1.

Good and bad about tools used



RQ1: Reasons for use or underuse

Tool output

- false positives
- number of warnings
- causing developer overload

Collaboration

- lack of or weak support for team collaboration
 - no easy way to share settings
 - FindBugs provide useful, but not enough feature
-

RQ1: Reasons for use or underuse

Customizability

- many tools are not easy to configure
- do not accommodate customizations developers want

Result understandability

- poor results/analysis presentation
- not enough information to help developers on
 - **what** the problem is
 - **why** it is a problem
 - what they should be doing **different**

RQ2: Workflow integration

Sometimes a developer's process includes running a tool

- it is not part of his/her workflow
- stop and run a tool in the middle of a process
- "disjoint process"

Developers prefer tools to run in background

- to fit into their development workflow
- IntelliJ looks like more "real time"

RQ2: Workflow integration

Some developers do not use IDEs.

Static analysis tools need:

- to be compiler integrated
- (users dislike go out of their coding environment)

For C/C++:

- Clang is integrated with compiler
 - Coverity and Lint are not capable of being integrated
-

RQ2: Workflow integration

Some tools **prevents** developers from being **productive**.

- taking long time to run

Occurs even with IDE integrated tools

- FindBugs: too slow
- IntelliJ: although utilizes *idle time*, but is still bothersome

RQ2: Workflow integration

Users prefer to fix bugs

- Once they are introduced
 - context to understand the bug is in developer's working memory
 - recommended strategy
- Later in development process
 - developer must recall the context

RQ3: Tool design

Main goal of this research: **improve** static analysis tools for developers.

Improvements gathered from users responses:

- quick-fix design
- "warning" notification and manipulation design
- other good ideas

1- Quick-fix design

Interactive quick fixes

- easy access to refactoring
- semi-automatically helping users to solve warnings

Negative points

- full quick-fix could be hasty and
- cause more problems

2- Warning notification & manipulation design

All 20 participants told when and how they want to be notified of errors in their code:

Fast!!

- faster feedback
- in an efficient way that
- does not interrupt their workflows

2- Warning notification & manipulation design

Some participants want the tool:

- **integrated** with IDE

Other ones:

- running the tool together at **build** and **compile** time
- results being presented when they are at a "*stopping point*"

2- Warning notification & manipulation design

Quickness should be accompanied with **discretion!**

Suggestions for warning manipulations:

- setting **aside** to view later
- temporary **suppression**
- possibility to **ignore** each individual defect
- **save** judgements and **share** them with other developers
- treating each warning as a **Mylin task**

3- Other design ideas

Editor plasticity:

- possibility to **surround** the affected code with the information expanded from a warning
- **holistic** view of the project's bugs instead of a list
 - pie-chart
 - heat-maps
- **gradients** of one color to indicate severity (instead of multiple distinct)

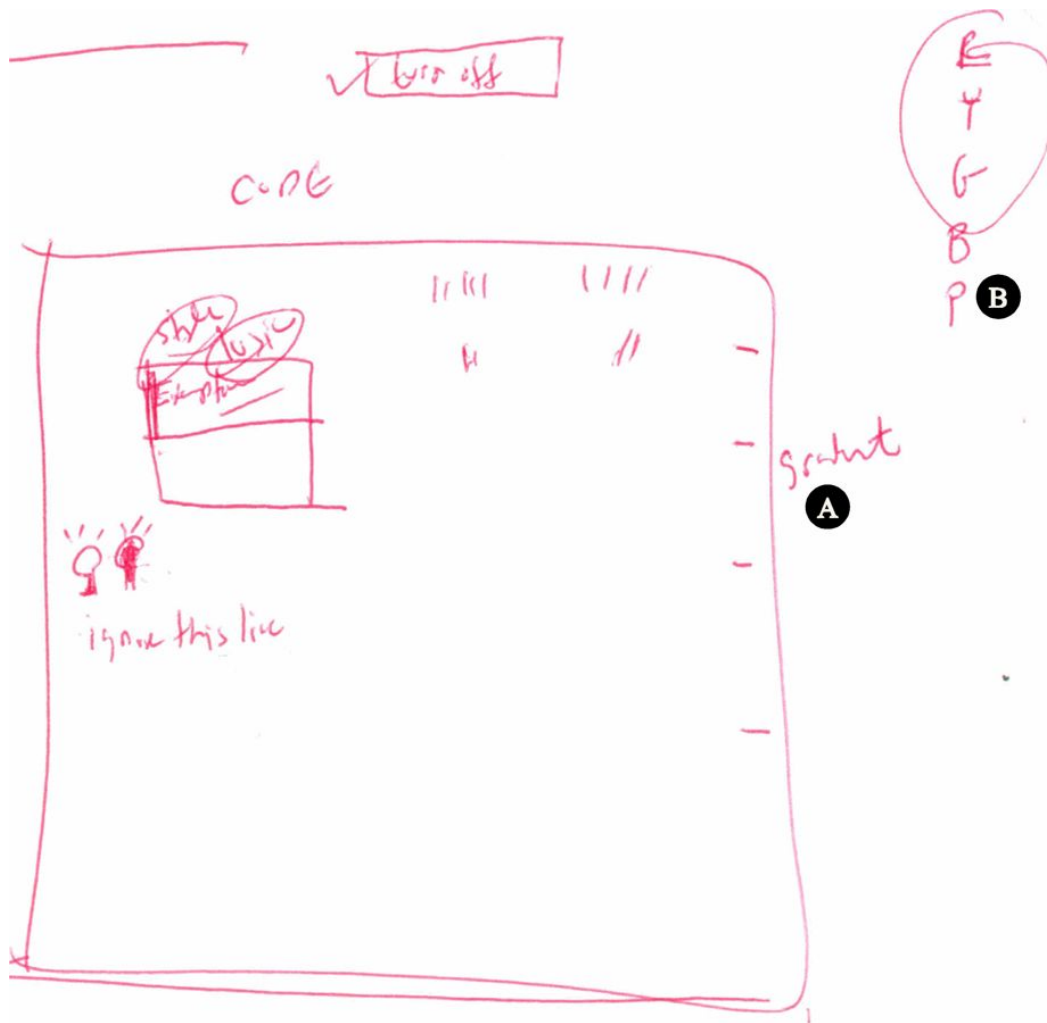
Example of a drawing

A

gradient

B

Colors to indicate severity



Threats to validity

External

Only 20 interviews (no generalization)

- valuable information
- Layman study with same methodology and 18 participants

Only developers who used static analysis tools.

Internal

The way authors conducted remote interviews:

For confidentiality reasons, not all participants could use their own workstation during interactive interview.

Workaround:

- Log4J and Ant projects
- FindBugs tool

Valuable information even from remote participants.

Construct

Limitation: developers were not always

- as **familiar** with the code or environment they had to use
- as with their own ones

Unintended hints in consent forms:

- participants could respond to questions based on what they **think** the researchers want to 'hear'

Final remarks

Future Work

The authors have begun to implement a tool with all improvements discovered in this study.

Run a study to evaluate their prototype.

Discussion and Conclusion

- Discussion basically summarizes Results section
- Conclusion summarizes Discussion!

Questions?

Why Don't Software Developers Use Static Analysis Tools to Find Bugs?

Thank you!!!

Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and
Robert Bowdidge
